

Final Project: Dynamic Breaker Enhancement for Power System Simulator

Michael Bliesath

May 1, 2026

1 Project Overview

The objective of this final project was to enhance the existing Python-based power system simulator by adding dynamic breaker functionality. The final project directions required a simulator enhancement that integrates cleanly with the existing solver framework, demonstrates modular object-oriented design, includes testing, and provides analytical value for power system studies. The project also required documentation, a validation plan, and at least two test cases with expected results.

The enhancement implemented in this project adds support for circuit breakers that can open or close connections in the simulated power system. Breakers are important because they allow the simulator to represent switching events, equipment outages, system islanding, and changes in network topology. Instead of hardcoding breaker behavior for one specific five-bus case, the updated implementation allows breakers to work dynamically for any system built using the simulator.

2 Purpose of the Enhancement

Before this enhancement, the simulator could model buses, generators, loads, transformers, transmission lines, admittance matrices, power mismatch equations, and Newton-Raphson-related calculations. However, the network topology was mostly fixed after the circuit was created. This meant that once a transmission line or transformer was added, it was always included in the Ybus matrix.

The breaker enhancement changes this by allowing system elements to be disconnected or reconnected. If a breaker is closed, the connected element remains active and contributes to the Ybus matrix. If a breaker is opened, the associated connection is removed from the active network. This allows the same simulator to analyze normal operating conditions, line outages, transformer outages, generator disconnections, load disconnections, and islanded systems.

3 Code Additions and Modifications

3.1 Breaker Class

A new `Breaker` class was added to represent a switching device. Each breaker stores its name, connection information, status, and optional equipment association. The breaker can be opened, closed, or toggled. A closed breaker means the connected element remains active, while an open breaker means the connected element is removed from the active circuit model.

The breaker class was designed to support both bus-to-bus connections and element-based connections. This makes the code flexible because a breaker can control a transmission line, transformer, generator, load, or direct bus-to-bus connection.

```
class Breaker:
    def __init__(self, name, node1_name=None, node2_name=None,
                  is_closed=True, rating=0.0,
                  element_type=None, element_name=None):
        self.name = name
        self.node1_name = node1_name
        self.node2_name = node2_name
        self.is_closed = is_closed
        self.rating = rating
        self.element_type = element_type
        self.element_name = element_name

    def open(self):
        self.is_closed = False

    def close(self):
        self.is_closed = True

    def toggle(self):
        self.is_closed = not self.is_closed
```

3.2 Circuit Class Updates

The `Circuit` class was updated to store breaker objects in a dictionary. A new `add_breaker()` method was added so that breakers can be added to the system like other equipment classes. This follows the same object-oriented structure used for buses, loads, generators, transformers, and transmission lines.

The `calc_ybus()` method was also modified so that it checks breaker status before stamping a transformer or transmission line into the Ybus matrix. If a breaker controlling a branch is open, that branch is skipped and does not contribute to the system admittance matrix.

```
self.breakers = {}

def add_breaker(self, name, node1_name=None, node2_name=None,
                is_closed=True, rating=0.0,
```

```

        element_type=None, element_name=None):
    breakerobj = Breaker(name, node1_name, node2_name,
                        is_closed, rating,
                        element_type, element_name)
    self.breakers[name] = breakerobj
    return breakerobj

```

This allows the simulator to dynamically change network topology without rewriting the system model.

3.3 Dynamic Ybus Behavior

The Ybus matrix is the core network representation used for power flow calculations. Since breakers change which elements are connected, breaker status must directly affect Ybus construction. The updated logic checks whether each transformer or transmission line is connected before stamping its primitive admittance matrix.

Conceptually, the logic is:

- If the breaker is closed, stamp the element into Ybus.
- If the breaker is open, skip that element.
- Recalculate Ybus after each topology change.

This makes the simulator capable of representing outages and switching operations.

4 Generator and Load Updates

The project also required dynamic handling of generators and loads. If a generator breaker is opened, that generator should no longer contribute real power to the system. If the disconnected generator was the slack generator, the circuit must select another active generator as the new slack source if one is available.

An `update_generator()` method was added to update generator participation and bus types dynamically. This method checks which generators are still connected, determines whether a slack generator still exists, and reassigns bus types when necessary. This prevents the solver from relying on a disconnected slack generator.

Loads were also made breaker-aware. If a load breaker is opened, that load no longer contributes to the specified power mismatch calculation.

5 Adjacency List and Island Detection

An adjacency list method was added to track the current network connectivity. The adjacency list stores which buses are connected to each other through closed transmission lines and transformers. This is useful because opening breakers can isolate buses or split the system into islands.

The adjacency list supports:

- identifying active buses,
- detecting islanded buses,
- preventing isolated buses from being solved incorrectly,
- supporting future improvements such as island-specific slack assignment.

```
def adjacency_list(self):
    adjacency = {bus_name: [] for bus_name in self.buses.keys()}

    for line in self.transmission_lines.values():
        if self.is_transmission_line_connected(line):
            adjacency[line.bus1_name].append(line.bus2_name)
            adjacency[line.bus2_name].append(line.bus1_name)

    for transformer in self.transformers.values():
        if self.is_transformer_connected(transformer):
            adjacency[transformer.bus1_name].append(transformer.
                bus2_name)
            adjacency[transformer.bus2_name].append(transformer.
                bus1_name)

    return adjacency
```

6 Testing Strategy

The project directions require a clear testing strategy with typical use cases, edge cases, and failure modes. The tests created for this enhancement include a `system_test.py` file and a `solver_test.py` file. These tests build the five-bus validation system and then apply different breaker operations to verify that the simulator responds correctly.

6.1 System Test

The system test focuses on topology behavior. It checks whether opening and closing breakers correctly changes the active network. For example, opening a transmission line breaker should remove that line from Ybus and update the adjacency list. Closing the breaker should restore the connection.

The system test validates:

- breaker creation,
- breaker open and close behavior,
- active bus detection,
- islanded bus detection,
- Ybus changes after breaker switching.

6.2 Solver Test

The solver test checks how breaker changes affect power flow behavior. It uses the five-bus system and runs several cases, including the base case, a line outage case, a transformer outage case, a generator disconnection case, and a load disconnection case.

These tests validate that:

- the base case solves normally,
- opening a line breaker changes Ybus,
- opening a generator breaker removes generator contribution,
- opening a load breaker removes load demand,
- slack generator logic updates when needed.

7 Validation Plan

The validation plan follows the final project requirements by comparing expected simulator behavior against known power system principles and PowerWorld-style validation. The base five-bus system is first tested with all breakers closed. This represents the normal operating condition and should match the previously validated circuit model.

After the base case is confirmed, individual breakers are opened one at a time. The expected result is that the associated element no longer contributes to Ybus or power mismatch calculations. For example, if a transmission line breaker is opened, the off-diagonal admittance terms associated with that line should disappear from the Ybus matrix. If a load breaker is opened, the load should no longer subtract real and reactive power from the bus power specification.

This validation approach demonstrates that the breaker implementation is not simply hardcoded for one case, but instead works dynamically based on the objects stored in the circuit.

8 Inputs and Outputs

The primary inputs to the breaker enhancement are:

- breaker name,
- controlled buses or controlled element,
- breaker status,
- optional breaker rating.

The primary outputs are:

- updated Ybus matrix,
- updated active bus list,
- updated islanded bus list,
- updated power mismatch vector,
- updated generator and bus type assignments.

9 How to Run the Tests

To run the system-level breaker tests, use:

```
python system_test.py
```

To run the solver-level breaker tests, use:

```
python solver_test.py
```

Each test prints the case being evaluated and the resulting circuit behavior. The expected behavior is that the base case runs normally, while breaker operations produce corresponding changes in Ybus, active buses, islanded buses, and solver behavior.

10 Conclusion

This project successfully enhances the Python power system simulator by adding dynamic breaker functionality. The new breaker implementation integrates with the existing object-oriented simulator framework and allows transmission lines, transformers, generators, and loads to be disconnected or reconnected during simulation.

The enhancement improves the analytical value of the simulator because it allows users to study outages, topology changes, islanding behavior, and switching events. The added test scripts verify that the implementation works for the five-bus validation case and can be extended to larger systems. Overall, the breaker enhancement satisfies the final project requirement to add a modular, well-tested simulator feature that supports power flow analysis.

References

1. J. D. Glover, M. S. Sarma, and T. Overbye, *Power System Analysis and Design*.
2. PowerWorld Corporation, *PowerWorld Simulator Documentation*.
3. ECE 1774/2774 Course Materials, University of Pittsburgh.